

PulseAgent: Autonomous Real-Time Intelligence & Reporting Platform

A Blueprint for Automated Data Ingestion, LLM Synthesis, and Multi-Channel Delivery

Author:	Product Management & AI Architecture	Target Engine:	Cursor IDE (LLM Code Generation)
Version:	1.0 (Production-Ready)	Status:	Approved for Development
Date:	June 2, 2026	Focus:	High UI/UX Fidelity & System Cleanliness

1. Executive Summary & Vision

PulseAgent is an autonomous, 24/7 background intelligence platform designed to bridge the gap between static LLM knowledge bases and the rapidly changing live internet. The system automates the manual loop of searching, filtering, reading, and summarizing daily industry news, financial streams, or topic-specific updates.

Unlike consumer chat tools that require manual prompting, PulseAgent acts as an unattended background service (orchestrated via an intelligent cron schedule) that pulls text data via external APIs, handles complex prompt engineering contexts natively, outputs a highly polished report, and distributes it directly to stakeholders where they already work (Slack, Email, Discord).

💡 Core UI/UX Goal: Eliminate the "Black Box" Anxiety

Since the backend engine executes automatically while the user sleeps, the user interface must prioritize extreme visibility. It must clearly visualize the **Pipeline State** (Idle, Fetching, Synthesizing, Delivering), provide human-readable logs of past automated runs, and present a seamless mechanism to "Trigger Instantly" or tweak prompts without breaking underlying data models.

2. Functional Requirements & Pipeline Architecture

The system consists of a web-based dashboard configuration panel and an autonomous backend processing pipeline split into four key sequential modules.

2.1 Pipeline Stages (The Backend Flow)

Stage	Functional Responsibility	Failure Resolution Mode
1. The Scheduler	Time-based cron engine (e.g., AWS EventBridge / local cron) wakes up the worker instance at specific, user-defined intervals (e.g., every day at 07:00 UTC).	Log omission to database; retry 3 times with exponential backoff before sending alert.
2. Data Ingestion	Connects to configured APIs (NewsAPI, Google Custom Search, CoinGecko, custom web scrapers). Fetches raw payloads, filters duplicates, strips raw HTML noise, and structures text into a clean JSON array.	If an API is down, use cached data or fall back to secondary web-search provider. Mark "Partial Ingestion" in UI.

Stage	Functional Responsibility	Failure Resolution Mode
3. LLM Processing	Injects structured text payloads into pre-defined prompt templates. Utilizes models like Gemini 1.5 Pro or GPT-4o via API to synthesize information, resolve conflicting reports, and generate Markdown text.	Graceful error handling for context-window overflows by chunking payloads dynamically.
4. Delivery	Parses LLM output, applies branding rules, and sends payloads to chosen webhooks (Slack block kits, Discord embeds, or SendGrid HTML email templates).	If a webhook fails, save output to local history and flag the run as "Delivery Failed" in dashboard.

3. UI/UX & Interaction Design

The user interface must be clean, responsive, and data-dense, drawing inspiration from modern developer portals (like Vercel or Supabase) but designed for business analysts and product operators.

3.1 High-Fidelity Layout Hierarchy

- **Global Navigation:** Simple left-hand sidebar detailing [Agents Dashboard], [Global API Integrations], [Activity Logs], and [System Settings].
- **Agent Dashboard (Main View):** Cards tracking individual active reporting bots. Each card displays Name, Target Subject, Frequency, Status Badge (Active, Paused, Running), and Date of the last report generated.
- **Interactive Pulse Indicator:** A visual line map at the top of an active agent's page demonstrating which stage of the 4-step pipeline is executing in real-time if a manual run is initiated.

3.2 Specific Interaction States & Design Standards

- **Human-Friendly Cron Configurator:** Instead of forcing raw expression string inputs (e.g., `0 7 * * *`), the UI must provide a simple dropdown for interval selection (Daily, Weekly, Hourly) paired with local timezone-adjusted clock selectors.
- **Live Markdown Preview Pane:** When editing the System Prompt or defining report frameworks, a split-screen view must render live placeholder data so the user can see exactly how the LLM will structure section headers, bulleted lists, and tables.
- **Citation Transparency Accordion:** Every generated report stored in the application must include an expandable "Sources Audited" section. Clicking an item must reveal the exact URLs, timestamps, and snippets of text pulled by the ingestion engine. This establishes trust and allows quick validation.

 **UI Color Palette Blueprint**

- **Primary Canvas:** Pure Clean White (#ffffff) with subtle slate borders (#e2e8f0) to match professional productivity tools.
- **Accents & Focus states:** Deep Royal Indigo (#2563eb) for buttons, primary toggles, and processing state tracks.
- **Semantic Badges:** Success/Delivered (#10b981 light tint), Processing/Synthesizing (#3b82f6 pulse animation), Failed (#ef4444 light tint).

4. Technical Specifications & Data Models

To ensure this PRD can be effortlessly translated into modular code by Cursor, use the following interface declarations as the single source of truth for schema design.

4.1 Database Types & Relationships (TypeScript Blueprint)

```
interface AgentConfiguration {
  id: string;
  name: string; // e.g., "Daily Competitor Intelligence"
  topicKeywords: string[]; // e.g., ["LLM hardware", "NVIDIA H200"]
  cronSchedule: string; // Clean standard cron format stored on backend
  systemPrompt: string; // Engineering constraints injected into LLM
  status: 'ACTIVE' | 'PAUSED' | 'RUNNING';
  dataSources: {
    sourceType: 'NEWS_API' | 'GOOGLE_SEARCH' | 'CUSTOM_SCRAPE';
    apiEndpoint: string;
    authSecretKeyRef: string; // Pointer to encrypted environment variable
  }[];
  deliveryChannels: {
    target: 'SLACK' | 'EMAIL' | 'DISCORD';
    webhookUrl: string;
    recipientList?: string[]; // Used if target is EMAIL
  }[];
}

interface IntelligenceReport {
  id: string;
  agentId: string;
  timestamp: string; // ISO 8601 execution marker
  rawIngestedDataCount: number;
  generatedMarkdown: string; // Core synthesized payload
  status: 'SUCCESS' | 'PARTIAL_FAILURE' | 'CRITICAL_ERROR';
  sourcesUsed: {
    title: string;
    url: string;
    snippet: string;
  }[];
}
```

5. Security, Constraints & Non-Functional Requirements

5.1 Rate Limiting & Cost Control

- **API Guardrails:** The Ingestion engine must cap maximum tokens extracted per external request to prevent astronomical upstream API bills. If a scraped article exceeds 8,000 words, it must pass through an extractive truncation step before hitting the primary LLM Synthesis layer.
- **Execution Timeout:** Any execution loop that hangs during ingestion or LLM completion for longer than 180 seconds must kill the worker thread automatically, write a detailed breakdown to the error log, and release system resources.

5.2 Secrets & Key Governance

Users must never paste plain-text vendor production keys (OpenAI, Anthropic, Google Search API) directly into unencrypted application databases. The UI must provide a dedicated secure setup interface, mapping values to secure local server environment files (.env) or modern key vaults.

Cursor Implementation Prompting Note

When parsing this document to construct the frontend application, generate the views layout iteratively. Build the **Agent Dashboard Grid View** first, hook it up to the Mock Database State using the AgentConfiguration interface above, and implement the custom UI components before proceeding to backend pipeline hookups.